QUESTION 1:

CODE:

1.cpp

```cpp
#include <iostream>
using namespace std;


char toLower(char ch) {
    if (ch >= 'A' && ch <= 'Z') {
        return ch + 32;
    }
    return ch;
}

// Function to check if character is a letter
bool isLetter(char ch) {
    return (ch >= 'A' && ch <= 'Z') || (ch >= 'a' && ch <= 'z');
}

int countRepeatedLetters(char* word) {
    int freq[26] = {0};

    // Count frequency of each letter
    int i = 0;
    while (word[i] != '\0') {
        if (isLetter(word[i])) {
            char ch = toLower(word[i]);
            freq[ch - 'a']++;
        }
        i++;
    }

    // Count total repeated letters
    int count = 0;
    for (int i = 0; i < 26; i++) {
        if (freq[i] > 1) {
            count += (freq[i] - 1);
        }
    }

    return count;
}

int main() {
    int size;

    cout << "Enter the maximum size of the string: ";
    cin >> size;

    // Dynamically allocate memory for the string
    char* str = new char[size];

    cout << "Enter a string: ";
    cin.getline(str, size);

```

```
    // Allocate memory for result word
    char* maxWord = new char[size];
    maxWord[0] = '\0';
    int maxRepeated = 0;

    int start = 0;
    for (int i = 0; i <= size; i++) {
        // Found end of word or end of string
        if (str[i] == ' ' || str[i] == '\0' || !isLetter(str[i])) {

            int wordLen = i - start;
            if (wordLen > 0) {
                char* word = new char[wordLen + 1];

                for (int j = 0; j < wordLen; j++) {
                    word[j] = str[start + j];
                }
                word[wordLen] = '\0';

                // Count repeated letters
                int repeated = countRepeatedLetters(word);
                if (repeated > maxRepeated) {
                    maxRepeated = repeated;


                    for (int j = 0; j <= wordLen; j++) {
                        maxWord[j] = word[j];
                    }
                }

                delete[] word;
            }

            start = i + 1;
        }

        if (str[i] == '\0') break;
    }

    // Print result
    if (maxRepeated > 0) {
        cout << "Word which has the highest number of repeated letters: " << maxWord << endl;
    } else {
        cout << "No word with repeated letters found." << endl;
    }

 //free memory
    delete[] str;
    delete[] maxWord;

    return 0;
}
```

CODE:2

```cpp
#include <iostream>
using namespace std;

int getLength(char* str) {
    int len = 0;
    while (str[len] != '\0') {
        len++;
    }
    return len;
}

// Function to check if subseq is a subsequence of main string
int isSubsequence(char* mainStr, char* subseq) {
    int mainLen = getLength(mainStr);
    int subLen = getLength(subseq);

    int i = 0;
    int j = 0;

    // Traverse both strings
    while (i < mainLen && j < subLen) {
        if (mainStr[i] == subseq[j]) {
            j++;
        }
        i++;
    }

    // If all characters of subseq are found, return 1
    if (j == subLen) {
        return 1;
    }
    return 0;
}
```

```
int main() {
    int size1, size2;

    cout << "Enter the maximum size for main string: ";
    cin >> size1;
    cin.ignore(); // Clear newline

    // Dynamically allocate memory for main string
    char* mainStr = new char[size1];

    cout << "Enter the main string: ";
    cin.getline(mainStr, size1);

    cout << "Enter the maximum size for subsequence: ";
    cin >> size2;
    cin.ignore();

    // Dynamically allocate memory for subsequence
    char* subseq = new char[size2];

    cout << "Enter the subsequence: ";
    cin.getline(subseq, size2);

    // Check if subseq is a subsequence of mainStr
    int result = isSubsequence(mainStr, subseq);

    cout << "Output: " << result << endl;
```


```
    cout << "Output: " << result << endl;
```


```

    // Free memory
    delete[] mainStr;
    delete[] subseq;

    return 0;
}
```

OUTPUT:

```
Enter the maximum size for main string: 50
Enter the main string: apple
Enter the maximum size for subsequence: 20
Enter the subsequence: ppe
Output: 1

--------------------------------
Process exited after 63.97 seconds with return value 0
Press any key to continue . . .
```

2.

```
C:\Users\MNA\OneDrive\Desl

Enter the maximum size for main string: 50
Enter the main string: APPLE
Enter the maximum size for subsequence: 20
Enter the subsequence: LAP
Output: 0

--------------------------------
Process exited after 16.43 seconds with return value 0
Press any key to continue . . .
```

PROBLEM 3:

```cpp
#include <iostream>
using namespace std;


int* sortArray(int *arr, int size) {
    int* result = new int[size];

    // Count odd and even numbers
    int oddCount = 0;
    int evenCount = 0;

    for (int i = 0; i < size; i++) {
        if (*(arr + i) % 2 != 0) {
            oddCount++;
        } else {
            evenCount++;
        }
    }

    // Create temporary arrays for odd and even
    int* oddArr = new int[oddCount];
    int* evenArr = new int[evenCount];

    int oddIndex = 0;
    int evenIndex = 0;

    for (int i = 0; i < size; i++) {
        if (*(arr + i) % 2 != 0) {
            *(oddArr + oddIndex) = *(arr + i);
            oddIndex++;
        } else {
            *(evenArr + evenIndex) = *(arr + i);
            evenIndex++;
        }
    }

    for (int i = 0; i < oddCount - 1; i++) {
        for (int j = 0; j < oddCount - i - 1; j++) {
            if (*(oddArr + j) > *(oddArr + j + 1)) {
                int temp = *(oddArr + j);
                *(oddArr + j) = *(oddArr + j + 1);
                *(oddArr + j + 1) = temp;
            }
        }
    }
```

```
    for (int i = 0; i < evenCount - 1; i++) {
        for (int j = 0; j < evenCount - i - 1; j++) {
            if (*(evenArr + j) > *(evenArr + j + 1)) {
                int temp = *(evenArr + j);
                *(evenArr + j) = *(evenArr + j + 1);
                *(evenArr + j + 1) = temp;
            }
        }
    }

    int resultIndex = 0;
    for (int i = 0; i < oddCount; i++) {
        *(result + resultIndex) = *(oddArr + i);
        resultIndex++;
    }
    for (int i = 0; i < evenCount; i++) {
        *(result + resultIndex) = *(evenArr + i);
        resultIndex++;
    }

    // Free temporary arrays
    delete[] oddArr;
    delete[] evenArr;

    return result;
}

// Function to print array using pointer notation
void printArray(int* arr, int size) {
    for (int i = 0; i < size; i++) {
        cout << *(arr + i);
        if (i < size - 1) {
            cout << ", ";
        }
    }
    cout << endl;
}
```

```
int main() {
    int size;

    cout << "Enter the size of the array: ";
    cin >> size;

    // Dynamically allocate memory for the array
    int* arr = new int[size];

    cout << "Enter " << size << " elements: ";
    for (int i = 0; i < size; i++) {
        cin >> *(arr + i);  // Using pointer notation
    }

    cout << "\nOriginal array: ";
    printArray(arr, size);

    // Sort the array
    int* sortedArr = sortArray(arr, size);

    cout << "Sorted array: ";
    printArray(sortedArr, size);

    // Free memory
    delete[] arr;
    delete[] sortedArr;


     return 0;
}
```

OUTPUT:

```
Enter the size of the array: 7
Enter 7 elements: 1 2 4 3 5 7 6

Original array: 1, 2, 4, 3, 5, 7, 6
Sorted array: 1, 3, 5, 7, 2, 4, 6

--------------------------------
Process exited after 15.3 seconds with return value 0
Press any key to continue . . .
```

CODE: 4

```cpp
#include <iostream>
using namespace std;


int* resizeArray(int* arr, int& capacity) {
    int newCapacity = capacity * 2;

    // Allocate new array with larger capacity
    int* newArr = new int[newCapacity];

for (int i = 0; i < capacity; i++) {
        newArr[i] = arr[i];
    }

    // Delete old array
    delete[] arr;

    // Update capacity
    capacity = newCapacity;

    cout << "Array resized! New capacity: " << capacity << endl;

    return newArr;
}

// Function to print array
void printArray(int* arr, int size) {
    cout << "Array elements: ";
    for (int i = 0; i < size; i++) {
        cout << arr[i];
        if (i < size - 1) {
            cout << ", ";
        }
    }
    cout << endl;
}

int main() {
    int initialCapacity;

    cout << "Enter initial capacity of the array: ";
    cin >> initialCapacity;

    // Dynamically allocate initial array
    int* arr = new int[initialCapacity];
    int capacity = initialCapacity;
    int size = 0; // Current number of elements

    cout << "Enter integers (enter -1 to stop):" << endl;
```

```
    cout << "Enter integers (enter -1 to stop):" << endl;

    int value;
    while (true) {
        cout << "Enter value: ";
        cin >> value;

        // Check for sentinel value
        if (value == -1) {
            break;
        }

        // Check if array is full
        if (size == capacity) {
            cout << "Array is full! Resizing..." << endl;
            arr = resizeArray(arr, capacity);
        }

        // Add element to array
        arr[size] = value;
        size++;

        cout << "Element added. Current size: " << size << "/" << capacity << endl;
    }

    // Display final array
    cout << "\n--- Final Array ---" << endl;
    cout << "Total elements: " << size << endl;
    cout << "Final capacity: " << capacity << endl;
    printArray(arr, size);

    // Free memory
    delete[] arr;

    return 0;
}
```

OUTPUT:

```
C:\Users\MNA\OneDrive\Desl

Enter initial capacity of the array: 4
Enter integers (enter -1 to stop):
Enter value: 1
Element added. Current size: 1/4
Enter value: 2
Element added. Current size: 2/4
Enter value: 3
Element added. Current size: 3/4
Enter value: 4
Element added. Current size: 4/4
Enter value: 5
Array is full! Resizing...
Array resized! New capacity: 8
Element added. Current size: 5/8
Enter value: 6
Element added. Current size: 6/8
Enter value: 7
Element added. Current size: 7/8
Enter value: 8
Element added. Current size: 8/8
Enter value: -1

--- Final Array ---
Total elements: 8
Final capacity: 8
Array elements: 1, 2, 3, 4, 5, 6, 7, 8

--------------------------------
Process exited after 24.59 seconds with return value 0
Press any key to continue . . .
```

CODE 5:

```cpp
#include <iostream>
using namespace std;

void findFrequency(int* arr, int size, int* unique, int* freq, int& uniqueCount) {
    uniqueCount = 0;

    for (int i = 0; i < size; i++) {
        bool found = false;

        for (int j = 0; j < uniqueCount; j++) {
            if (unique[j] == arr[i]) {
                freq[j]++;
                found = true;
                break;
            }
        }

        if (!found) {
            unique[uniqueCount] = arr[i];
            freq[uniqueCount] = 1;
            uniqueCount++;
        }
    }
}

void sortBySum(int* unique, int* freq, int uniqueCount) {
    for (int i = 0; i < uniqueCount - 1; i++) {
        for (int j = 0; j < uniqueCount - i - 1; j++) {
            int sum1 = unique[j] * freq[j];
            int sum2 = unique[j + 1] * freq[j + 1];

            if (sum1 < sum2) {
                int temp = unique[j];
                unique[j] = unique[j + 1];
                unique[j + 1] = temp;

                temp = freq[j];
                freq[j] = freq[j + 1];
                freq[j + 1] = temp;
            }
        }
    }
}

int maximizeSumWithKDistinct(int* arr, int size, int k) {
    if (k == 0 || size == 0) {
        return 0;
    }

```

```
    int* unique = new int[size];
    int* freq = new int[size];
    int uniqueCount = 0;

    findFrequency(arr, size, unique, freq, uniqueCount);
    sortBySum(unique, freq, uniqueCount);

    int maxSum = 0;
    int distinctCount = 0;

    for (int i = 0; i < uniqueCount && distinctCount < k; i++) {
        maxSum += unique[i] * freq[i];
        distinctCount++;
    }

    delete[] unique;
    delete[] freq;

    return maxSum;
}

void printArray(int* arr, int size) {
    cout << "Array: ";
    for (int i = 0; i < size; i++) {
        cout << arr[i];
        if (i < size - 1) {
            cout << ", ";
            cout << ", ";
        }
    }
    cout << endl;
}

int main() {
    int size;

    cout << "Enter the size of the array: ";
    cin >> size;

    int* arr = new int[size];

    cout << "Enter " << size << " elements: ";
    for (int i = 0; i < size; i++) {
        cin >> arr[i];
    }

    int k;
    cout << "Enter the value of K (maximum distinct integers): ";
    cin >> k;

    cout << "\n";
    printArray(arr, size);
```

```
    cout << "K = " << k << endl;

    int maxSum = maximizeSumWithKDistinct(arr, size, k);
    cout << "\nMaximum sum with at most " << k << " distinct integers: " << maxSum << endl;

    delete[] arr;

    return 0;
}
```

OUTPUT:

```
C:\Users\MNA\OneDrive\Desl

Enter the size of the array: 5
Enter 5 elements: 10 20 30 40 50
Enter the value of K (maximum distinct integers): 3

Array: 10, 20, 30, 40, 50
K = 3

Maximum sum with at most 3 distinct integers: 120

--------------------------------
Process exited after 137.7 seconds with return value 0
Press any key to continue . . .
```